

拓扑 AC | CSP-S & NOIP 联测数据结构专题讲评

袁胜利

2026 年 8 月 21 日

吐槽环节/自由分享

坐标折叠

对每个前缀，把已经出现的不同数从小到大编号。询问一个数 x 从首次出现开始，在各个前缀中的排名之和。

$$S(x) = \sum_{j=f(x)}^n g_j(x),$$

其中 $f(x)$ 是首次出现位置， $g_j(x)$ 是 x 在前缀 $a_1, \dots, a_j$ 中的排名。
 $n, q \leq 2 \times 10^5$, $a_i \leq 10^8$ 。

一个数是否已经进入前缀，只看首次出现位置。因此

$$g_j(x) = 1 + \#\{y < x \mid f(y) \leq j\}.$$

x 自身从 $f(x)$ 到 n 始终贡献 1。每个比 x 小的 y 要等到 x, y 都出现后才会让排名增加，所以每个 y 的贡献为 $n - \max(f(x), f(y)) + 1$ 次。

坐标折叠

子任务一（30 分， $n \leq 5000$ ）：先求每个不同值的首次出现位置。
对每个 x 枚举所有更小的不同值 y ，累加 $n - \max(f(x), f(y)) + 1$ ，最后补上 x 自身的贡献。
预处理时间复杂度为 $O(n^2)$ ，询问 $O(q)$ 。

观察

按数值从小到大处理 x ，此时数据结构中恰好只有比 x 小的值。
把更小值分成两类：

- 首次位置不超过 $f(x)$ ：每个值贡献 $n - f(x) + 1$ ；
- 首次位置大于 $f(x)$ ：每个值贡献 $n - f(y) + 1$ 。

第一棵树状数组查询首次位置不超过 $f(x)$ 的数的数量 cnt 。设此时所有数的总数量为 i 。

第二棵树状数组查询首次位置不超过 $f(x)$ 的数的和 sum 。设此时所有数的总和为 tot 。

则答案为 $(cnt + 1) * (n - f(x) + 1) + [(i - cnt) * (n + 1) - (tot - sum)]$ 。
算完后，在位置 $f(x)$ 分别在两棵树状数组插入数量 $f(x)^0$ 和权值 $f(x)^1$ 。

坐标折叠

子任务二 (50 分, $a_i \leq 10^5$):

直接用数值本身作下标, 求出每个值的首次出现位置, 再从 1 到 10^5 扫描值域。

扫描到 x 时, 两棵数状数组中恰好保存所有更小值, 直接套用上一页的转移。

时间复杂度为 $O(n + 10^5 \log n + q)$ 。

子任务三 (20 分, 无额外限制):

使用离散化将 a_i 缩小到和 n 同阶。

时间复杂度为 $O(n \log n + q)$ 。

极值商和

给定两两不同的正整数序列。对每个非空连续子数组，定义权值

$$\left\lfloor \frac{\max a_i}{\min a_i} \right\rfloor,$$

求全部子数组的权值和。

$n \leq 3 \times 10^5$, $a_i \leq 3 \times 10^5$ 。

极值商和

子任务一 (30 分, $n \leq 300$): 枚举左端点 l , 向右移动 r 时滚动维护区间最小值与最大值, 直接累加答案。

时间复杂度为 $O(n^2)$ 。

子任务二 (20 分, $\max a_i \leq 5000$): 注意到由于所有元素互不相同, 所以 $n \leq 5000$ 。

继续使用平方枚举, 时间复杂度为 $O(n^2)$ 。

笛卡尔树

给定一个元素两两不同的数组。它的小根笛卡尔树是满足下面两个条件的二叉树：

- 中序遍历的结点顺序与原数组下标顺序相同；
- 每个结点的值都小于它的两个儿子的值。

这两个条件唯一确定一棵树。因为中序遍历顺序不变，每个结点的子树都对应原数组中的一段连续区间；该结点就是这段区间的最小值。

笛卡尔树可以用单调栈在 $O(n)$ 时间内建立。按下标从左到右扫描：

- 弹出栈中所有值大于当前值的结点；
- 最后弹出的结点成为当前结点的左儿子；
- 若栈仍非空，当前结点成为栈顶结点的右儿子；
- 将当前结点压入栈中。

观察

固定结点 i 。以 a_i 为最小值的子数组必须包含 i ，并且完全落在 i 的笛卡尔子树内。

若左右子树大小分别为 l_i, r_i ，这类子数组共有

$$C_i = (l_i + 1)(r_i + 1)$$

个。

一个常见的计数转化

若一个子数组的最小值为 x 、最大值为 M ，则

$$\left\lfloor \frac{M}{x} \right\rfloor = \sum_{q \geq 1} [M \geq qx].$$

所以每个以 x 为最小值的子数组先贡献一次；最大值每达到 $2x, 3x, \dots$ ，再各贡献一次。

问题变成在已知 $a_i = x$ 为最小值，枚举 $q > 0$ ，然后对阈值 $t = qx \leq n$ ，统计最大值至少为 t 的数组数量。这样的枚举是调和级数的。

子任务三 (20 分, $\max a_i \leq 10^5$): 对每个出现的 x 枚举 $2x, 3x, \dots$ 。
在一种数据结构 (如线段树) 上寻找 i 左右最近的、值不小于阈值 t 的位置。再与 i 的笛卡尔子树区间取交, 就能算出最大值小于 t 的子数组数。
调和级数为 $O(n \log n)$, 每次边界查询为 $O(\log n)$ 。总时间复杂度为 $O(n \log^2 n)$ 。

观察

满分做法把所有的 $n \log n$ 个阈值 t 从小到大排序，按顺序处理。回答阈值 t 前，激活所有值小于 t 的位置，并把相邻激活位置用并查集合并。位置 i 所在连通块 $[u, v]$ ，是包含 i 且所有元素都小于 t 的最大连续段。笛卡尔子树区间为 $[i - l_i, i + r_i]$ ，两者取交：

$$p = \max(i - l_i, u), \quad q = \min(i + r_i, v),$$

则此时满足最小值要求却不满足最大值要求的区间数量为：

$$D(i, t) = (i - p + 1)(q - i + 1).$$

先把全部 C_i 加入答案。对每个出现的值 x ，把查询挂到阈值 $2x, 3x, \dots$ ；扫描到 t 时使答案增加 $C_i - D(i, t)$ 。

极值商和

子任务四 (30 分, 无额外限制): 每个子数组只会归到笛卡尔树中代表其最小值的结点。它在阈值 $x, 2x, \dots, \lfloor M/x \rfloor x$ 各被计数一次, 总次数正好是 $\lfloor M/x \rfloor$ 。

单调栈可以直接求出每个笛卡尔子树的左右边界, 不必显式保存树。总时间复杂度为 $O(n + n \log n + n\alpha(n))$ 。

树上中继

树上点 i 的频率为 w_i 。若 z 位于 u, v 的路径上, 且

$$w_u - w_z \leq P, \quad w_v - w_z \leq P,$$

则 z 是无序点对 $\{u, v\}$ 的中继。求最大的整数 $P \in [-10^6, 10^6]$, 使每个点只能成为严格小于 K 对无序点的中继。

$N \leq 3 \times 10^5, W \leq 10^6$ 。

观察

固定中继 z ，把条件改写为点权阈值 $x = w_z + P$ 。端点 u 合格当且仅当 $w_u \leq x$ 。

把树根定为 1，记：

- a ：全树合格端点数；
- s ： z 子树中的合格端点数；
- c_i ： z 的第 i 个儿子子树中的合格端点数。

则

$$\text{ans}_z(x) = s(a - s) + \binom{s}{2} - \sum_i \binom{c_i}{2}.$$

前面统计 z 子树与父亲方向之间的点对。后面先统计子树内部点对，再删去落在同一个儿子子树内的点对。

若 $P < 0$ ，可能有 $w_z > x$ ，此时 z 本身不会被计入 s ，但公式仍然成立。随着 x 增大，合格端点只会增加，负载单调不减。对每个 z ，可以寻找最小的失效阈值 x_z ，使

$$\text{ans}_z(x_z) \geq K.$$

树上中继

子任务一 (30 分, $N \leq 1000$): 对每个候选中继 z 二分点权阈值 x 。
一次判定遍历整棵树, 统计删去 z 后各个方向中权值不超过 x 的点数,
再代入负载公式。
每个点做 $O(\log W)$ 次全树判定, 时间复杂度为 $O(N^2 \log W)$ 。

观察

子树在 DFS 序中是一段连续区间。按 DFS 序依次插入点权，建立权值主席树。任意子树内权值不超过 x 的点数，都能由两个版本之差得到。对固定 z ，需要同时维护这些版本对：

- 全树区间；
- z 的子树区间；
- z 的每个儿子子树区间。

若外层二分 x ，每次再对所有区间做主席树查询，会产生两个值域对数。可以让这些版本对直接同步向值域叶子下降。

子任务二 (50 分, $N \leq 10^5$): 保留外层二分。对每个候选中继 z , 二分最小失效阈值; 每次用主席树查询全树、 z 子树和各儿子子树中的合格点数。

一次判定的工作量为 $O((son(z) + 1) \log W)$, 其中 $son(z)$ 指的是 z 的儿子个数。对所有 $son(z)$ 的和显然为 $n - 1$, 因此总时间复杂度为 $O(N \log^2 W)$ 。

这一做法可以通过前两个子任务, 共获得 80 分。

树上中继

可以实现把二分线段树转化为线段树二分，具体为：

在主席树当前值域结点 $[l, r]$ ，读取每个版本对左儿子的计数，再加上此前已经越过的较小权值计数，就得到阈值 mid 下的 a, s, c_i 。

- 若负载已经达到 K ，最小失效阈值位于左半，所有版本对一起走左儿子；
 - 若负载仍小于 K ，把左半计数累入前缀，所有版本对一起走右儿子。
- 每一层只读取一次各区间计数，而不是重新完成一次区间查询。

子任务三 (20 分, 无额外限制): 对点 z , 每下降一层的工作量为 $O((z) + 1)$; 对所有 z 求和后, 每层总工作量为 $O(N)$ 。
若找到最小失效阈值 x_z , 该点允许的最大参数为

$$P_z = x_z - 1 - w_z.$$

若加入所有点后负载仍小于 K , 令 $P_z = 10^6$ 。最终答案为 $\min_z P_z$ 。
总时间复杂度为 $O(N \log W)$ 。

独行分组

把序列划分为若干非空连续段。每一段中，出现次数恰好为 1 的数不能超过 p 种，求合法划分数。

$1 \leq p \leq n \leq 10^5$ ，答案对 $10^9 + 7$ 取模。

观察

令 $f[i]$ 表示前 i 个位置的合法划分数，规定 $f[0] = 1$ 。若最后一段是 $[j+1, i]$ ，记其中只出现一次的数有 $s[i][j]$ 种，则

$$f[i] = \sum_{j=0}^{i-1} f[j] [s[i][j] \leq p].$$

转移的难点是：右端点从 $i-1$ 移到 i 时，怎样同时更新所有 $s[i][j]$ ，并求出满足阈值的 $f[j]$ 权值和。

独行分组

子任务一 (30 分, $n \leq 1000$): 固定右端点 i , 从 i 向左枚举最后一段并维护各值频次。

- 频次从 0 变成 1, 计数加一;
- 频次从 1 变成 2, 计数减一;
- 频次继续增加, 计数不变。

每当计数不超过 p , 就把对应的 $f[j]$ 加入 $f[i]$ 。时间复杂度为 $O(n^2)$ 。

观察

设新加入的 $a[i]$ 上一次、上上次出现的位置为 u, v , 不存在时记为 0。从右端点 $i-1$ 推进到 i , 只有 $a[i]$ 的贡献会改变:

- $v \leq j < u$: $s[i][j] = s[i-1][j] - 1$;
- $u \leq j < i$: $s[i][j] = s[i-1][j] + 1$;
- 其余位置满足 $s[i][j] = s[i-1][j]$ 。

所以考虑滚动数组, 每轮只有两次区间修改:

$$[v, u-1] -= 1, \quad [u, i-1] += 1.$$

独行分组

子任务二 (20 分, $p = 1$): 固定当前右端点 i , 用线段树维护 $s[i][j]$, 在线段树每个结点中记录最小值/次小值/最小值对应的 f 的和/次小值对应的 f 的和。

区间加让最小值次小值一起平移, 使用普通懒标记; 合并儿子时, 从四个候选中保留最小的两个不同值。

注意到满足 $s[i][j] \leq 1$ 的状态只可能是根结点保存的最小值或次小值, 读出对应权值和即可。

时间复杂度为 $O(n \log n)$ 。

独行分组

子任务三 (30 分, $n \leq 5 \times 10^4$): 把切分位置按约 $\sqrt{n}$ 的块长分块。固定右端点 i , 块内按 $s[i][j]$ 排序, 并维护排序后的 $f[j]$ 的前缀和。整块区间加只修改懒标记; 边界块逐点修改后重新排序。查询所有 $s[i][j] \leq p$ 的权值和时, 在每个块内二分并读取前缀和。总时间复杂度为 $O(n\sqrt{n}\log n)$ 。

注意到如果把块大小设置为 $\sqrt{n\log n}$ 的话, 查询方法不变, 而修改边界块一定是修改一段前缀或后缀, 因此可以归并排序, 这样总复杂度为 $O(n\sqrt{n\log n})$, 应该卡不掉。

独行分组

子任务四 (20 分, 无额外限制):

考虑优化上面那个分块, 现在块大小仍为 $\sqrt{n}$, 我们现在已经知道了 $O(\sqrt{n})$ 修改零散块的方法, 因此只需要讨论如何对整块进行区间加/减一以及如何快速查询整块。

注意到 p 是永远不变的, 因此可以对于每一个块定义一个指针, 指向排序后的数组里 p 的位置。

我们惊人的发现只要把所有 s 值相同的位置看成一个整体, 那么区间加减一只会使得指针移动一个位置。因此对于一个整块, 查询修改均为 $O(1)$ 。

每轮耗时 $O(\sqrt{n})$, 总时间复杂度为 $O(n\sqrt{n})$ 。